

Laser Vault Heist

Computer Graphics Fundamentals

Final Project Report

Students: Nurbek Yerbulekov, Makhmet Demesh, Nurassyl Nygmet

Group: SE-2428

Topic 12
C++17 and OpenGL 3.3 Core

Table of Contents

1 Introduction 4

1.1 Team responsibilities 5

2 Literature Review and Background 5

2.1 Lighting and material response 5

2.2 Skinning and motion representation 6

2.3 Collision and visibility 7

2.4 Display mapping, shadows and textures 8

3 Project Description 9

3.1 Objective and problem statement 9

3.2 Scope and environment 9

3.3 Player controls and interactions 10

4 Technical Implementation 11

4.1 Languages, tools and build 11

4.2 Development and structure 11

4.3 Transformations and collision 12

4.4 Camera, input and frame pacing 13

4.5 Character and animation 14

5 Graphics and Rendering 16

5.1 Pipeline and live CCTV 16

5.2 Lighting and shading 17

5.3 Textures and post-processing 18

5.4 Failure, safe opening and finale 19

6 Testing and Evaluation 21

6.1 Verification method 21

6.2 Performance evaluation 21

6.3 Issues found and corrected 23

7 Conclusion and Future Work 24

8 References 25

8.1 Asset and technical credits 27

8.2 Journal ranking provenance 27

9 Appendix 29

9.1 Requirement coverage 29

9.2 Reproduction 29

9.3 Defense preparation 29

1 Introduction

Laser Vault Heist is a real-time 3D stealth game built around a mansion vault. The player observes six moving laser barriers, walks or crouches through the room and opens a mechanical safe. Touching a beam triggers a red alarm and a retry. The completed version adds first- and third-person cameras, a locked entrance corridor, a skinned thief and a one-shot dance finale. The application renders through its own OpenGL pipeline rather than a game engine.

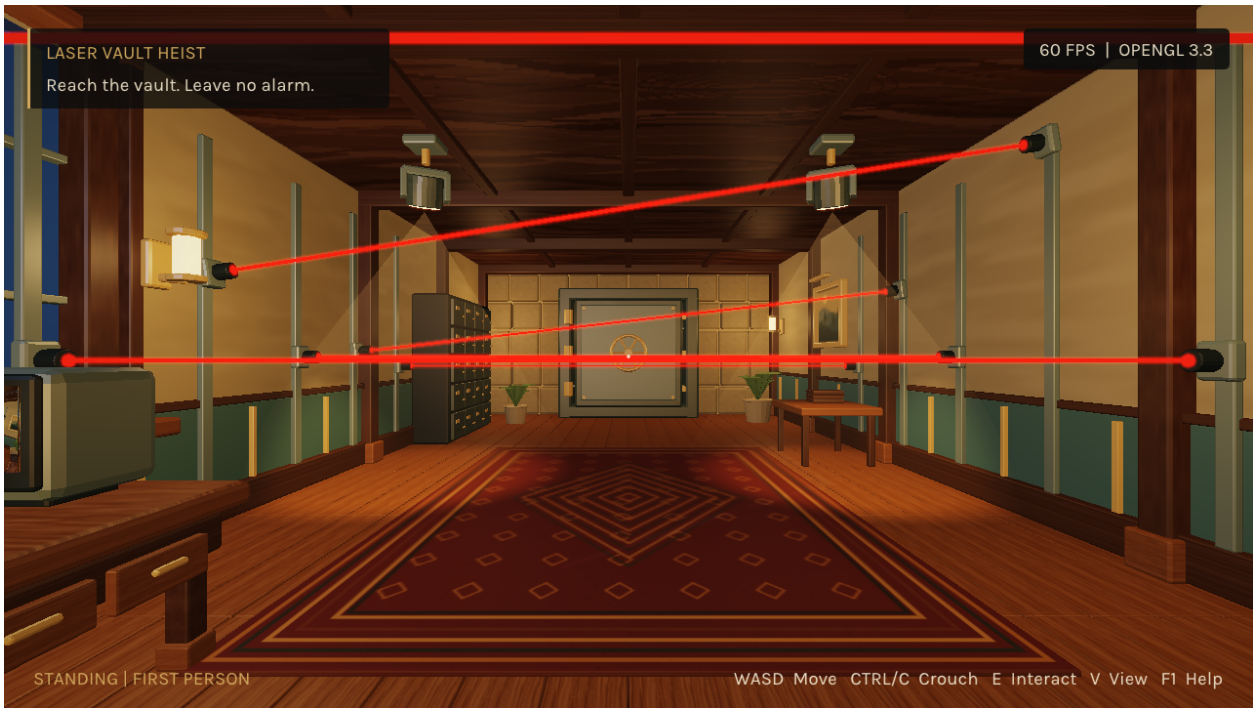


Figure 1. Current vault rendered by the application, with textured furniture, lasers and live CCTV

Figure 1 shows the current room rather than the earlier prototype. The project brings together indexed geometry, model/view/projection transformations, hierarchical animation, per-fragment lighting, texture sampling and interactive state changes. These systems have to agree: a laser must collide where it is drawn, crouching must fit the visible character, and a camera must not look through a wall.

This report follows the supplied course template. It compares graphics methods with eighteen journal articles, explains the implemented algorithms, and presents new captures and measurements from the current Windows executable. Tests and local measurements support the engineering claims; performance on the unknown TA machine and a live oral defense remain separate checks.

1.1 Team responsibilities

Student	Responsibility
Nurbek Yerbulekov	Project lead: game design, visual direction, models, animation and visual integration.
Makhmet Demesh	Gameplay mechanics: laser contact, progression and game states; rendering and technical integration.
Nurassyl Nygmet	Gameplay mechanics: controls and interactions; testing, build verification and documentation.

The visual pipeline and gameplay systems were developed together: the character proportions, animation poses, hazard clearance and state feedback had to remain consistent. Each team member should be prepared to explain the complete project as well as their assigned area.

2 Literature Review and Background

The review uses eighteen journal articles with DOI records retained in docs/references-verified.json. Sixteen remain after excluding the two Communications of the ACM entries. The accompanying source audit records the journal categories and public reproductions of SJR 2025 quartiles. Those rankings describe journals in that ranking year, not the historical quartile of every publication. Direct access to SCImago and Scopus was unavailable during the recorded audit, so direct verification of their databases is not claimed.

Journal	Recorded SJR category / quartile
ACM Transactions on Graphics	Computer Graphics and Computer-Aided Design; Q1
Computer Graphics Forum	Computer Graphics and Computer-Aided Design; Q1
IEEE Transactions on Visualization and Computer Graphics	Computer Graphics and Computer-Aided Design; Q1
Computers & Graphics	Computer Graphics and Computer-Aided Design; Q2
Communications of the ACM	Computer Science (miscellaneous); Q1

2.1 Lighting and material response

Phong et al. [1]. Phong compares shading rules and relates image quality to normals, hidden-surface removal and perception. The project evaluates lighting per fragment. It uses the Blinn half-vector variant rather than copying Phong's original reflection-vector expression.

Cook et al. [2]. Cook and Torrance model reflection using surface microgeometry and material-dependent directional response. This is the physical comparison for the vault's brass and painted metal. The implemented empirical Blinn-Phong shader is cheaper and easier to explain, but it does not reproduce a calibrated metal BRDF.

Schlick et al. [3]. Schlick develops a compact reflectance model that balances physical constraints with practical cost. Its efficiency goal supports keeping material controls understandable. The game does not implement the full Schlick BRDF or claim physically based rendering.

Phong-style per-fragment evaluation is a shading strategy, while Cook-Torrance and Schlick address reflectance. The project chooses empirical Blinn-Phong for legibility and explicit controls. Its brass appearance is an artistic approximation; the shader is not described as a physically calibrated conductor model.

2.2 Skinning and motion representation

Kavan et al. [4]. Kavan and colleagues replace linear matrix blending with approximate dual-quaternion blending to reduce collapsing-joint artifacts. The thief uses linear blend skinning because its weights and motions are simple. This paper identifies a concrete future improvement for shoulder and knee volume preservation.

Kavan et al. [5]. Kavan, Sloan and O'Sullivan fit standard linear skinning to animated meshes with sparse weights and efficient optimization. The runtime keeps at most four normalized influences per vertex and a compact matrix palette. It imports the supplied skin weights and validates them rather than fitting them with this optimization.

Le et al. [6]. Le and Deng jointly recover rigid bone transforms and sparse convex skinning weights from example poses. Weight non-negativity and normalization are checked during export. The supplied skeleton is retained; no joint/weight recovery algorithm is implemented.

Merry et al. [16]. Merry, Marais and Gain generalize skeletal subspace deformation while retaining a linear formulation and hardware rendering. The comparison explains why plain linear blend skinning can lose volume around strongly bent joints. Our constrained animation range and garment shapes mitigate, but do not mathematically eliminate, those artifacts.

Mohr et al. [17]. Mohr and Gleicher fit a compact real-time deformation model to examples exported from richer authoring tools. The project separates editable Blender resources from sampled local transforms and indexed runtime meshes. The supplied rig and derived clips are exported directly rather than fitted from a physically simulated character.

James et al. [18]. James and Twigg derive skinning approximations from deformable mesh sequences using automatically chosen transforms and weights. The current model has 52 weighted bones plus required ancestors, within the 64-matrix shader palette. No automatic discovery of bones or reconstruction of mesh sequences is performed.

Linear blend skinning offers a small and inspectable runtime. Dual quaternions and fitted deformation methods address stronger quality constraints, but would require different validation. The crouch export instead lowers the pelvis by 0.14 m and uses two-bone leg IK to preserve limb lengths and sole contact before sampling local transforms.

2.3 Collision and visibility

Klosowski et al. [7]. Klosowski and colleagues organize discrete-orientation bounding volumes into hierarchies for efficient collision queries in complex environments. A hierarchy is unnecessary for six laser boxes and a small set of furniture colliders. Transforming body samples into each beam's local space keeps the project inexpensive and directly inspectable.

Jiménez et al. [8]. Jimenez, Thomas and Torras classify continuous and sampled collision strategies and explain how temporal and spatial pruning control cost. The game adopts repeated static local-box tests at 240 Hz, with a conservative body margin. This is bounded sampling, not an exact continuous-collision solver for arbitrary velocities.

Teschner et al. [13]. Teschner and colleagues survey deformable collision using hierarchies, distance fields, spatial partitioning, image-space and stochastic methods. The animated thief is represented by a sampled upright body for gameplay. This avoids deforming triangle-level collision, while pose-bound checks keep the crouched mesh compatible with the chosen clearance.

Cohen-Or et al. [9]. Cohen-Or and colleagues survey visibility algorithms for interactive walkthroughs, including techniques that avoid drawing hidden geometry. The compact single room uses depth testing, back-face culling and static instancing. More complex occlusion systems would add overhead without a demonstrated benefit at this scene size.

Six oriented laser boxes and a few furniture proxies do not justify a full collision hierarchy. Fixed 240 Hz body sampling is inexpensive for the chosen movement speeds, while depth testing and static instancing handle the visual scene. This choice trades generality for a bounded problem rather than implementing the cited continuous or deformable solvers.

2.4 Display mapping, shadows and textures

Reinhard et al. [10]. Reinhard and colleagues adapt photographic tone reproduction to map high-dynamic-range images to a display. The project also renders into a floating-point target before display conversion. Its implemented global fitted curve is credited separately and is not Reinhard's photographic operator.

Larson et al. [11]. Larson, Rushmeier and Piatko preserve visibility through tone reproduction informed by luminance adaptation and human vision. This provides a criterion for tuning the night scene: the safe and hazards must remain readable. The game uses artistic tuning rather than the paper's histogram and visual-adaptation model.

Hasenfratz et al. [12]. Hasenfratz and colleagues compare real-time shadow methods by geometry, lighting conditions, image quality and cost. The vault uses one directional depth map with a small PCF kernel. The spotlights illuminate surfaces but do not each render a separate shadow map; their dust cones are an inexpensive visual approximation.

Blinn et al. [14]. Blinn and Newell use surface parameterization to map patterns and filter their contributions over a pixel footprint. The project's UV coordinates and mipmaps address the same sampling problem. Its mesh rasterizer does not reproduce the original patch renderer or environment-reflection system.

Lefebvre et al. [15]. Lefebvre and Hoppe synthesize controllable, deterministic textures through parallel neighborhood matching and multiresolution operations. Seeded environment textures are generated offline, while the thief uses the supplied base-color image. Neither uses this paper's neighborhood-matching synthesis at runtime.

Display readability motivates the HDR target, fitted tone curve and controlled laser glow. The glow filter now compares scene depth so distant emission does not spill over a nearer character. This is a screen-space visibility correction, not a physical volumetric scattering solution.

3 Project Description

3.1 Objective and problem statement

The objective is a complete implementation of topic 12 with all Core features and all three Stretch goals: rotating guard spotlights, live CCTV and safe-opening victory. A player must be able to fail, retry and win through the same rules shown in the demonstration. The technical problem is keeping rendered transforms, sampled collision, eye height, skinned motion and framebuffer outputs consistent.

3.2 Scope and environment

The vault measures approximately 8 by 15 by 4 metres. An enclosed entrance vestibule leads to a locked archive corridor. The room contains wood and stone finishes, a window, a desk, chair, local lamp, monitor, deposit drawers, framed art, plants, a rug and an articulated safe. A visible barred gate with a padlock and lockdown sign explains the blocked exit; the player and camera stop before its surface.

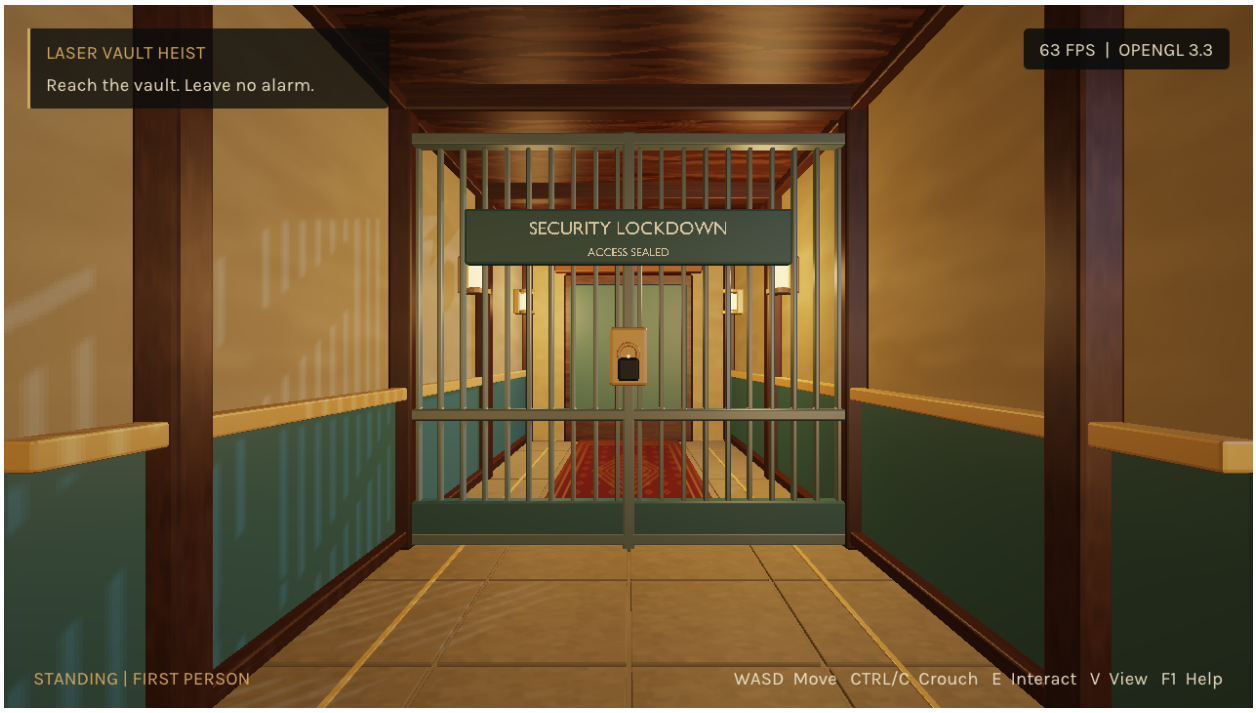


Figure 2. Enclosed entrance corridor and locked security gate replacing the former empty opening

Figure 2 shows a physical explanation for the movement limit. The decorative corridor continues behind the gate, but is outside the playable route. Painting frames, sconces and the relocated deposit cabinet are clear of the emitter hardware. Evaluated bounds are recorded in evidence/location-layout-validation.json.

One player, one vault route and one finale are included. Guard characters, combat, weapons, multiplayer, shops and extra levels are excluded. Sound is optional in the

brief and is not implemented. The source character and seven supplied actions came from the Meshy export; the skeleton names alone do not establish where each action was authored.

3.3 Player controls and interactions



Figure 3. Current menu with sensitivity and monitor-aware FPS controls

Figure 3 shows the controls available before play. The same settings are accessible while paused; the FPS display remains visible during play.

Control	Effect
WASD / mouse	Move relative to the view and look around
V	Switch first- and third-person view
Hold Ctrl or C	Crouch; lower the eye and body proxy
E	Use the lamp, CCTV selector or safe within 2.15 m
Esc / F1	Pause / controls help
R	Retry after alarm or victory

In first person the character follows the view heading. In third person the mouse orbits independently; standing movement turns the body toward actual displacement and plays forward walking even for A/D or S. Crouching preserves heading and selects the matching forward/back/side clip. The lamp changes illumination and glow, the monitor switches between two live views, and the safe requires a real crossing of the final beam.

4 Technical Implementation

4.1 Languages, tools and build

Component	Use
C++17 / GLSL 330	Gameplay, native input, resource ownership and custom GPU programs
GLFW / GLAD / GLM	Window/context, OpenGL loading and vector/matrix operations
Dear ImGui / stb	Menus/settings, PNG decode and framebuffer screenshots
Blender / Python	Model preparation, environment geometry, animation derivation and native export
CMake / LLVM-MinGW	Verified Windows Release build; dependencies are vendored
Python / FFmpeg	Evidence collection, PDF authoring and optional real-time recording

Python is an offline authoring and verification tool. The game does not execute Python, open Blender or contact an asset service at startup. Geometry, transforms and hierarchy are loaded from native scene data; camera matrices, rendering, lights, collision and animation playback are evaluated in C++ and GLSL. The course starter was not supplied, so exact compatibility with that unknown setup is not asserted. Windows x64 is tested; Linux, macOS and MSVC instructions are provided without claiming an executed build there.

4.2 Development and structure

The code separates window/input orchestration, fixed-step game state, scene loading, GPU resources, world animation, skinning, UI and capture. Static mesh/material groups are uploaded once and drawn with instancing. Dynamic nodes include safe components, moving security hardware and the character. The renderer performs shadow, CCTV, main HDR and display passes; the monitor is omitted from its own security pass to avoid framebuffer feedback.

Module	Responsibility
main.cpp / mouse_motion.hpp	Window, native mouse events, input consumption, orchestration
game.cpp / fixed_step.hpp	Player motion, contacts, interactions and state transitions
scene.cpp / graphics.cpp	Hierarchy, indexed meshes, textures, buffers and framebuffer ownership

Module	Responsibility
animation.cpp / world.cpp	Bone palette and transforms for safe, lasers, spots and lights
renderer.cpp / shaders	Shadow map, CCTV, surface shading and display conversion
ui.cpp / frame_limiter.hpp	Menus, sensitivity/FPS choices and steady-clock pacing

4.3 Transformations and collision

For column vectors the world matrix is the parent world matrix multiplied by the local transform. The model, view and projection sequence places geometry in clip space. Normals use an inverse transpose so that non-uniform scale does not rotate lighting incorrectly.

$$M_{world} = M_{parent} \times T \times R \times S$$
$$p_{clip} = P \times V \times M_{world} \times p_{local}$$
$$N_{world} = \text{normalize}(\text{transpose}(\text{inverse}(\text{mat3}(M_{world}))) \times N_{local})$$

Each moving beam provides the same model matrix to rendering and contact tests. A body point is transformed by the inverse beam matrix and compared against local half-extents. This avoids the false positives of a large world-aligned bounding box around a rotated beam.

$$q = \text{inverse}(M_{beam}) \times (p_{body}, 1)$$
$$\text{hit when } |q_x| \le h_x \text{ and } |q_y| \le h_y \text{ and } |q_z| \le h_z$$

The controller tests upright body samples at 0.08 m intervals, with conservative margins. Simulation advances at 1/240 s; one-shot interact/retry inputs remain pending until a step consumes them. Movement is normalized diagonally and tested against room/furniture proxies. This is discrete sampling for the specified speeds, not a continuous collision guarantee for arbitrary motion.

4.4 Camera, input and frame pacing

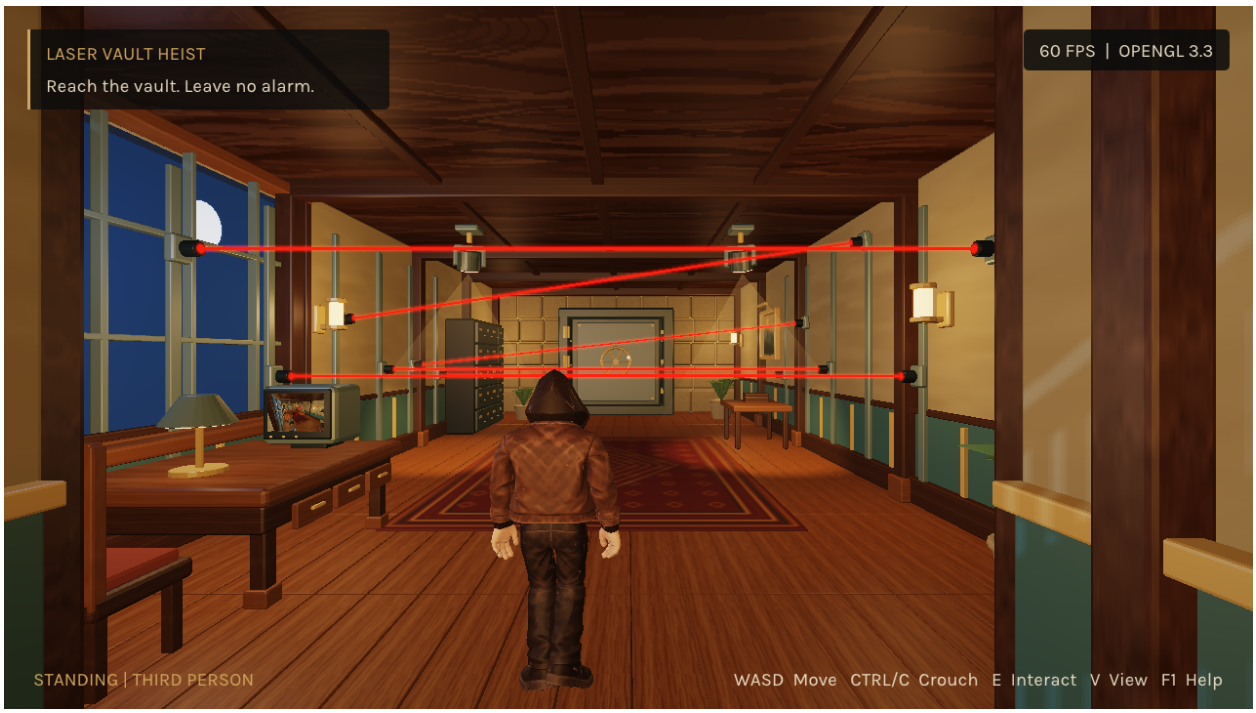


Figure 4. Third-person character view in the current vault

Figure 4 demonstrates the third-person view. Its boom is swept against padded scene proxies and refined near obstructions; it contracts before walls, furniture or the entrance gate. Changing boom distance does not change look direction. A camera very close to the hood hides the body in the main view while CCTV and shadows retain it.

Windows input uses WM_INPUT rather than repeatedly treating cursor position as fresh motion. Relative deltas are accumulated and consumed once per frame; absolute devices have separate per-device baselines. Focus/capture changes clear pending motion. Manual playtesting confirmed that this corrected the uncontrolled view drift. Mouse sensitivity is displayed on a Source-style multiplier scale using 0.022 degrees per count per unit; the previous physical default remains unchanged.

An FPS limiter follows the refresh rate of the monitor containing the window. Presets include reported monitor modes, custom entry accepts 1-10000 FPS, and Unlimited disables pacing. Retry preserves the selected view, sensitivity and cap. Windows uses a high-resolution waitable timer and a short final wait. Pacing changes render frequency; simulation and mouse-delta interpretation keep their own time/input rules.

4.5 Character and animation

The current thief has 30,725 triangles, 52 weighted bones plus required ancestors and no more than four normalized influences per vertex. The shader palette is limited to 64 matrices. The input FBX and source textures are preserved; runtime uses an indexed LVHM mesh, a scene manifest and sampled local TRS animation in LVHA. Blender coordinates (x,y,z) map once to runtime (x,z,-y).

$$p_{skinned} = \sum_{i=1}^{4} w_i \times B_i \times p_{bind}$$

$$\sum w_i = 1,\ w_i \geq 0;\ B_i = G_{pose,i} \times \text{inverse}(G_{bind,i})$$

Walking and running use supplied actions. Forward crouching reverses the supplied backward cycle; backward walking reverses Walking. Idle poses average opposing gait phases, and Interact adds a reach. Crouch export lowers the pelvis by 0.14 m and solves leg contact. Runtime selects clips from actual movement relative to body heading and advances gait phase by travelled distance.

Clip group	Runtime names
Standing	Idle, Walk, WalkBackward, Run
Crouching	CrouchIdle, CrouchWalk, CrouchBackward, CrouchLeft, CrouchRight
Actions	Interact, Dance, DanceLong



Figure 5. Current skinned character in a crouch-forward animation review

Figure 5 is rendered from the imported mesh and the derived forward clip. Tests skin actual runtime vertices over all crouch frames. The maximum checked crouch height is approximately 1.287 m, below the 1.33 m body proxy. Standing height is 1.78 m and eye heights are 1.57 m standing and 1.22 m crouched.

5 Graphics and Rendering

5.1 Pipeline and live CCTV



Figure 6. Live security camera showing the current animated character

Figure 6 shows the security view rendered into a 640 × 360 framebuffer. It contains the current room and animated thief, not a prerecorded image. A separate display target applies tone and gamma conversion for the enlarged UI view. The physical screen samples the linear security image as part of ordinary surface rendering.

Meshes use VAO/VBO/EBO and custom vertex/fragment programs. Instanced static objects share mesh/material groups. The main target is RGBA16F with a depth texture; the shadow target is a 2048 × 2048 depth map. Framebuffer completeness is checked. Depth testing and back-face culling reject hidden opaque fragments.

5.2 Lighting and shading



Figure 7. Desk-lamp interaction captured during actual gameplay

Figure 7 records the local lamp toggle: both the light contribution and visible glow follow its state. The other scene lights continue to operate.

Blinn-Phong shading is evaluated per fragment. Eight point lights cover the vault and corridor; two spotlights rotate with their equipment hierarchy. Ambient and directional moon terms establish the night palette. A lamp can be toggled. Alarm recolors illumination and emissive security fixtures red. The shader has capacity for twelve local lights.

$$L = ka * C + \Sigma \text{ attenuation} \times \text{cone} \times \text{lightColor} * [C \times \max(N \cdot L, 0) + ks \times \max(N \cdot H, 0)^{\wedge}n]$$

$$H = \text{normalize}(L + V)$$

$$\text{attenuation} = \text{power} / (1 + 0.20d + 0.095d^2)$$

The implemented equation uses a normalized half-vector and material shininess. Soft spotlight edges interpolate between inner and outer cone limits. A 3 × 3 PCF kernel filters the directional shadow map; the spotlights do not each have a shadow map. Additive dust-cone geometry is a visualization rather than volumetric light transport.



Figure 8. Rotating spotlight equipment and illumination in the actual runtime

Figure 8 connects the visible fixture hierarchy with the light direction used by the shader. Rotation is applied to the mount, and the light vector is derived from the same world transform. Hierarchical wheel, bolt and door transforms use the same parent/child principle.

5.3 Textures and post-processing

Environment assets use exported UV coordinates and seeded painted textures. The thief uses the supplied base-color image; additional source maps are retained without claiming that runtime implements normal/metallic/roughness PBR. PNGs are uploaded once, decoded from sRGB for lighting, and given generated mipmaps with trilinear minification.

A fitted filmic tone curve maps HDR color to display range, followed by gamma encoding and a restrained vignette. The curve coefficients are credited to Narkowicz [19]. Laser glow samples nearby bright pixels, but rejects samples farther than the current surface by a depth tolerance. Linear eye distance is reconstructed from the depth texture before that comparison.

$$\text{zndc} = 2 \times \text{depth} - 1$$
$$\text{deye} = 2 \times \text{near} \times \text{far} / (\text{far} + \text{near} - \text{zndc} \times (\text{far} - \text{near}))$$

This correction removed a measured 114-pixel glow spill onto the foreground character in the GPU regression fixture; the updated fixture reports zero such pixels. A foreground beam remains visible. The result demonstrates the tested occlusion case, not a universal promise of artifact-free rendering at every camera angle.

5.4 Failure, safe opening and finale



Figure 9. Crouched view below the last laser before the final passage

Figure 9 shows the crouched position below the last beam before crossing. The following passage uses the ordinary body proxy and contact tests; reaching the far side enables safe interaction.

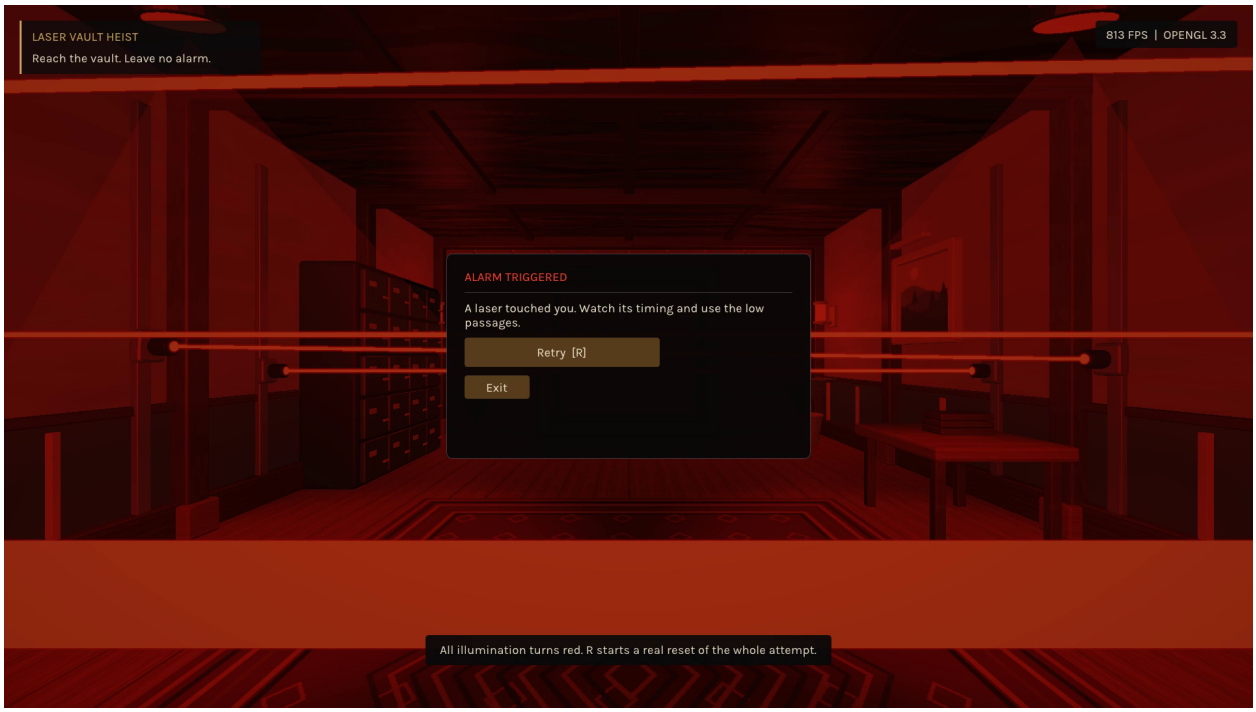


Figure 10. Actual alarm state following contact with a laser

Figure 10 shows a real failure under the same rules as the final route. Retry reconstructs initial game state and laser phases while retaining player settings. The safe remains locked until the player has actually crossed the last beam.

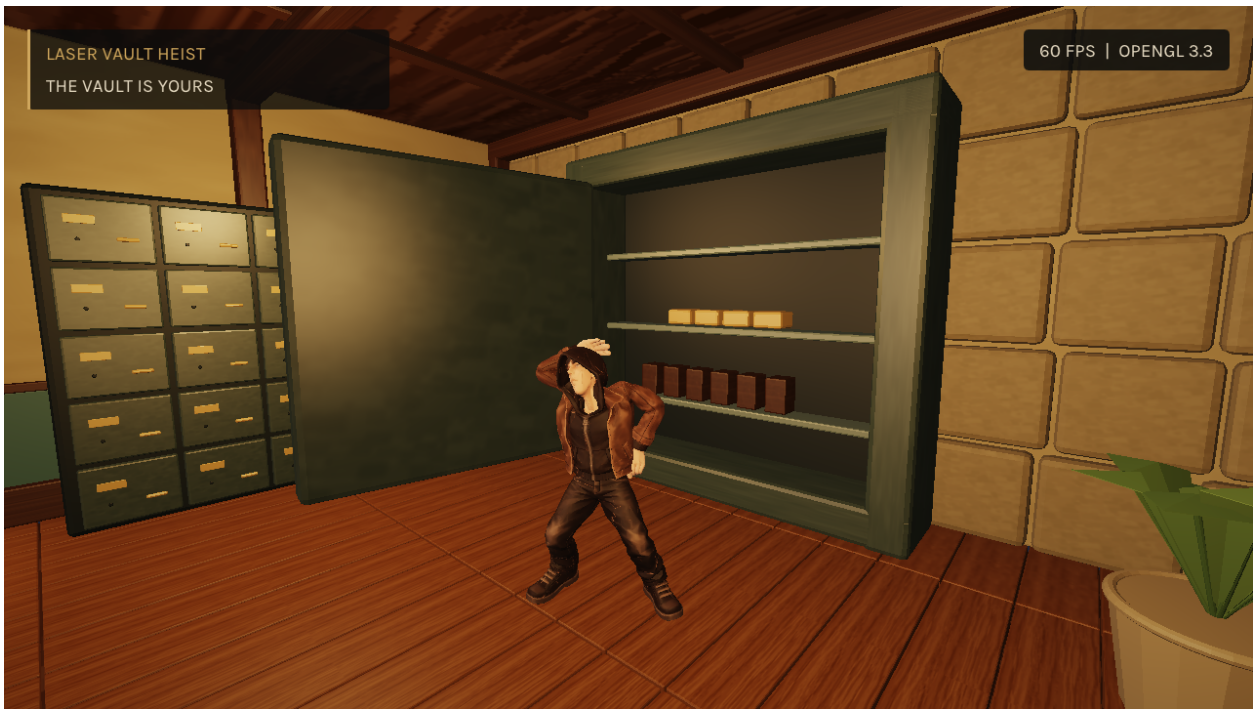


Figure 11. Open safe and the current character performing the one-shot victory dance

Figure 11 shows the finale. The wheel rotates, bolts retract and the door opens over 3.4 s. The character is revealed after the door sweep and plays the 7.5 s Dance clip once. Gameplay is locked during Finale, pause freezes it, and Victory follows the entire 10.9 s sequence. The room extension keeps the door at least 1.70 m clear of the last laser.



Figure 12. Victory appears after the complete safe-opening and dance sequence

Figure 12 confirms the completed state after the one-shot finale. The demonstration retains the final result rather than cutting to a fabricated success screen.

6 Testing and Evaluation

6.1 Verification method

CTest suite	Coverage
laser_character_occlusion	Real GPU baseline/behind/front beam comparisons; depth-aware glow; 2 cm inner-door/bolt clearance
gameplay_math	Oriented point-box contact, crouch clearance, states and interactions
complete_gameplay_route	Normal movement through six lasers and safe interaction
demo_and_slow_frames	Demo state progression, retained input edges, slow-frame consistency
imported_animation_clips	Actual runtime skinning, clip durations, weights and all crouch-frame bounds
complete_victory_cinematic	Door/dance duration, movement lock, pause, final frame and retry
player_camera_modes	Camera orbit, directional movement, native motion consumption, gate/prop obstruction, settings and complete third-person route

All seven tests passed on the current Windows build. New screenshots were captured from the executable, and the updated demonstration records real framebuffer frames. Its deterministic controller supplies ordinary movement and interaction inputs; it does not disable collision or teleport the player through the final route. Manual feedback also confirmed the corrected mouse behavior and the refreshed room. This feedback is a practical check, not a formal multi-user usability study.

6.2 Performance evaluation

Hardware: Intel Core i5-13400F, NVIDIA GeForce RTX 5060, approximately 32 GiB RAM, Windows x64. Runtime reports OpenGL 3.3.0 NVIDIA 617.42. Three new visible 1280 × 720 runs use the current scene, repeated legal routes, live CCTV and skinning. Each run lasts 25 s; the first 5 s are warm-up. VSync, FPS cap and recording are disabled during measurement.

Run	Avg FPS	1% low	p50 ms	p95 ms	p99 ms
1	1148.9	695.8	0.833	1.147	1.320
2	1185.3	726.5	0.810	1.081	1.265
3	1168.2	660.3	0.818	1.115	1.338

Average FPS is 1000 divided by mean frame time in milliseconds. The 1% low is 1000 divided by the mean of the slowest 1% of frame times. Percentiles use nearest rank. Timing covers the complete loop, including simulation, draw submission, UI, telemetry and presentation; it is not an isolated GPU timestamp. Desktop activity and OS scheduling can affect the tail.

Run	Frames measured	Peak working set	Peak private
1	22979	140.9 MiB	252.2 MiB
2	23705	144.9 MiB	254.1 MiB
3	23364	140.9 MiB	254.2 MiB

These figures describe this computer and this route only. The course target is evaluated on the TA machine, whose hardware has not been supplied. The FPS setting normally defaults to the active display refresh rate; the limiter was separately measured at 60, 144 and a custom 237 FPS. A cap is an upper bound and does not guarantee that expensive frames will meet it.

6.3 Issues found and corrected

Issue	Cause and correction
Uncontrolled mouse rotation	Cursor positions could be re-counted as motion. Windows WM_INPUT accumulation and one-time consumption replaced that path; device baselines and focus resets are explicit.
Laser visible across character edges	Screen-space bloom ignored foreground depth. Depth-aware rejection removed the measured spill.
Flicker inside the safe door	Retracted brass bolts ended exactly on the inner door plane. Their rear faces are now 2 cm inside the slab.
Decorative intersections	Frames and deposit furniture overlapped security rails. Paintings and cabinet were relocated; evaluated bounds show no targeted decoration/security overlaps.
Empty entrance	The opening exposed the background clear color. An enclosed corridor, visible locked gate and matching collision now close the route.
Safe door touching the last laser	The room and safe approach were extended; the full door sweep now has 1.70 m minimum clearance.

7 Conclusion and Future Work

The current implementation integrates all mandatory technical areas and all three topic stretch goals in a playable vault route. It uses custom buffers and shaders, model/view/projection transformations, articulated objects, dynamic per-fragment illumination, UV textures with mipmaps, keyboard/mouse controls, three interactions and a visible FPS counter. The updated visual scene and character are supported by source assets, reproducible exports and seven passing tests.

The project demonstrates that consistency across systems matters more than a feature list. Render transforms must match contact volumes; camera motion must consume fresh input only once; an emissive post-process must respect depth; and a mechanical assembly must avoid coincident surfaces. Recent fixes address each of these concrete failures and are represented in the new evidence.

Future work should first test the exact TA machine and rehearse the live presentation with all three members. Technical extensions could include continuous collision for higher speeds, more stable shadow filtering, stronger deformation for extreme poses and accessibility or sound settings. Additional levels and combat are outside the current scope. Actual oral understanding, each member's account history, Moodle submission and the assigned grade cannot be established by local automated checks.

8 References

[1] Bui Tuong Phong, "Illumination for computer generated pictures", Communications of the ACM, vol. 18, no. 6, pp. 311-317, 1975. DOI: 10.1145/360825.360839.

https://doi.org/10.1145/360825.360839

[2] R. L. Cook, K. E. Torrance, "A Reflectance Model for Computer Graphics", ACM Transactions on Graphics, vol. 1, no. 1, pp. 7-24, 1982. DOI: 10.1145/357290.357293.

https://doi.org/10.1145/357290.357293

[3] Christophe Schlick, "An Inexpensive BRDF Model for Physically-based Rendering", Computer Graphics Forum, vol. 13, no. 3, pp. 233-246, 1994. DOI: 10.1111/1467-8659.1330233.

https://doi.org/10.1111/1467-8659.1330233

[4] Ladislav Kavan, Steven Collins, Jiří Žára, Carol O'Sullivan, "Geometric skinning with approximate dual quaternion blending", ACM Transactions on Graphics, vol. 27, no. 4, pp. 1-23, 2008. DOI: 10.1145/1409625.1409627.

https://doi.org/10.1145/1409625.1409627

[5] L. Kavan, P.-P. Sloan, C. O'Sullivan, "Fast and Efficient Skinning of Animated Meshes", Computer Graphics Forum, vol. 29, no. 2, pp. 327-336, 2010. DOI: 10.1111/j.1467-8659.2009.01602.x.

https://doi.org/10.1111/j.1467-8659.2009.01602.x

[6] Binh Huy Le, Zhigang Deng, "Smooth skinning decomposition with rigid bones", ACM Transactions on Graphics, vol. 31, no. 6, pp. 1-10, 2012. DOI: 10.1145/2366145.2366218.

https://doi.org/10.1145/2366145.2366218

[7] J.T. Klosowski, M. Held, J.S.B. Mitchell, H. Sowizral, K. Zikan, "Efficient collision detection using bounding volume hierarchies of k-DOPs", IEEE Transactions on Visualization and Computer Graphics, vol. 4, no. 1, pp. 21-36, 1998. DOI: 10.1109/2945.675649.

https://doi.org/10.1109/2945.675649

[8] P. Jiménez, F. Thomas, C. Torras, "3D collision detection: a survey", Computers & Graphics, vol. 25, no. 2, pp. 269-285, 2001. DOI: 10.1016/S0097-8493(00)00130-8.

https://doi.org/10.1016/s0097-8493(00)00130-8

[9] D. Cohen-Or, Y.L. Chrysanthou, C.T. Silva, F. Durand, "A survey of visibility for walkthrough applications", IEEE Transactions on Visualization and Computer Graphics, vol. 9, no. 3, pp. 412-431, 2003. DOI: 10.1109/TVCG.2003.1207447.

https://doi.org/10.1109/tvcg.2003.1207447

[10] Erik Reinhard, Michael Stark, Peter Shirley, James Ferwerda, "Photographic tone reproduction for digital images", ACM Transactions on Graphics, vol. 21, no. 3, pp. 267-276, 2002. DOI: 10.1145/566654.566575.

https://doi.org/10.1145/566654.566575

[11] G.W. Larson, H. Rushmeier, C. Piatko, "A visibility matching tone reproduction operator for high dynamic range scenes", IEEE Transactions on Visualization and Computer Graphics, vol. 3, no. 4, pp. 291-306, 1997. DOI: 10.1109/2945.646233.

https://doi.org/10.1109/2945.646233

[12] J. -M. Hasenfratz, M. Lapierre, N. Holzschuch, F. Sillion, "A Survey of Real-time Soft Shadows Algorithms", Computer Graphics Forum, vol. 22, no. 4, pp. 753-774, 2003. DOI: 10.1111/j.1467-8659.2003.00722.x.

https://doi.org/10.1111/j.1467-8659.2003.00722.x

[13] M. Teschner, S. Kimmerle, B. Heidelberger, G. Zachmann, L. Raghupathi, A. Fuhrmann, M.-P. Cani, F. Faure, N. Magnenat-Thalmann, W. Strasser, P. Volino, "Collision Detection for Deformable Objects", Computer Graphics Forum, vol. 24, no. 1, pp. 61-81, 2005. DOI: 10.1111/j.1467-8659.2005.00829.x.

https://doi.org/10.1111/j.1467-8659.2005.00829.x

[14] James F. Blinn, Martin E. Newell, "Texture and reflection in computer generated images", Communications of the ACM, vol. 19, no. 10, pp. 542-547, 1976. DOI: 10.1145/360349.360353.

https://doi.org/10.1145/360349.360353

[15] Sylvain Lefebvre, Hugues Hoppe, "Parallel controllable texture synthesis", ACM Transactions on Graphics, vol. 24, no. 3, pp. 777-786, 2005. DOI: 10.1145/1073204.1073261.

https://doi.org/10.1145/1073204.1073261

[16] Bruce Merry, Patrick Marais, James Gain, "Animation space: a truly linear framework for character animation", ACM Transactions on Graphics, vol. 25, no. 4, pp. 1400-1423, 2006. DOI: 10.1145/1183287.1183294.

https://doi.org/10.1145/1183287.1183294

[17] Alex Mohr, Michael Gleicher, "Building efficient, accurate character skins from examples", ACM Transactions on Graphics, vol. 22, no. 3, pp. 562-568, 2003. DOI: 10.1145/882262.882308.

https://doi.org/10.1145/882262.882308

[18] Doug L. James, Christopher D. Twigg, "Skinning mesh animations", ACM Transactions on Graphics, vol. 24, no. 3, pp. 399-407, 2005. DOI: 10.1145/1073204.1073206.

https://doi.org/10.1145/1073204.1073206

[19] K. Narkowicz, "ACES Filmic Tone Mapping Curve", 2016. Fitted tone-mapping expression, CC0 option. https://knarkowicz.wordpress.com/2016/01/06/aces-filmic-tone-mapping-curve/

8.1 Asset and technical credits

The character source package contains the Meshy-exported thief, textures, skeleton and seven supplied motions. They are retained in art/source/animated_thief. Derived clips and retargeting are recorded in docs/animated-thief-integration.md. Bone names use the Mixamo convention, which is not proof that every motion came from Mixamo. The room, furniture, corridor and environment textures were created for this project with the retained Blender/Python scripts. The referenced game Just Another Night Shift is a style reference, not a source of extracted meshes or code.

Third-party components: GLFW (zlib/libpng), GLM (MIT option), Dear ImGui (MIT), stb (MIT option), generated GLAD declarations (CC0 plus registry/generator notices), and Karla by Jonathan Pinhorn (SIL OFL 1.1). Exact versions, upstream links and license copies are in CREDITS.md, docs/dependency-lock.json and LICENSES/. The supplied course documents and concept images remain local and are not republished as project artwork.

Primary technical sources include https://www.glfw.org/documentation.html, https://github.com/g-truc/glm, https://github.com/ocornut/imgui, https://github.com/nothings/stb, https://github.com/Dav1dde/glad and https://github.com/ValveSoftware/source-sdk-2013. The latter documents the Source mouse yaw/pitch coefficient; the project uses its own input implementation.

8.2 Journal ranking provenance

ACM Transactions on Graphics: SJR 2025 Q1, Computer Graphics and Computer-Aided Design.

https://www.journalsbase.com/journals/acm-transactions-on-graphics

https://journalindexes.com/acm-transactions-on-graphics-07300301/

Computer Graphics Forum: SJR 2025 Q1, Computer Graphics and Computer-Aided Design.

https://www.croris.hr/casopisi/casopis/10902

https://scienceaijournal.com/journals/computer-graphics-forum-0167-7055

IEEE Transactions on Visualization and Computer Graphics: SJR 2025 Q1, Computer Graphics and Computer-Aided Design.

https://www.iit.comillas.edu/publicacion/info_revista/en/912/ieee-transactions-on-visualization-and-computer-graphics

https://researchcloud.net/journal/1941-0506

Computers & Graphics: SJR 2025 Q2, Computer Graphics and Computer-Aided Design.

https://researchcloud.net/journal/0097-8493

https://akaturk.com/journals/25028?lang=en

Communications of the ACM: SJR 2025 Q1, Computer Science (miscellaneous).

https://www.iit.comillas.edu/publicacion/info_revista/en/16/Communications_of_the_ACM

https://scienceaijournal.com/journals/communications-of-the-acm-0001-0782

9 Appendix

9.1 Requirement coverage

ID	Implemented evidence
R1	C++17, OpenGL 3.3 core, VAO/VBO/EBO and separate GLSL programs
R2	Vault, corridor, furniture, security hardware, safe and animated character
R3	Custom MVP and parent/child transforms for safe and rotating equipment
R4	Per-fragment Blinn-Phong, toggleable lamp, moving spots and alarm
R5	UV-mapped environment/character textures with generated mipmaps
R6	Two camera modes, keyboard/mouse controls and three interactions
R7	Visible counter and current hardware/FPS table; TA-machine check remains
R8	Vendored CMake build and exact README commands; course starter unavailable

9.2 Reproduction

Build with the README commands and run CTest. Standard startup discovers assets and shaders beside the executable. Use --root PATH for an authoring checkout, --entrance-preview for the new gate view, --character-preview --clip CrouchWalk for the rig, and --finale-preview for the actual finale. --fps 0 disables the cap for benchmarks; --fps auto follows the monitor. The ordinary release needs no Python, Blender, compiler or network connection.

The report figure files are fresh runtime captures in evidence/screenshots/report-current. Raw current benchmark CSVs and the summary are in evidence/benchmarks-current. The previous benchmark folder remains historical. A release archive attached to an earlier tag may predate the latest source, so reviewers should use the current tagged release or build the branch identified in README.

9.3 Defense preparation

Nurbek should present the visual direction, model/animation pipeline and how the art stays consistent with gameplay. Makhmet should explain inverse-transform laser contact, state transitions and rendering stages. Nurassyl should explain fresh input consumption, interactions, tests, reproduction and measurements. This is a presentation plan, not a claim that the oral defense has already happened. Every member should also explain MVP, the Blinn half-vector, normalized skin weights, framebuffer feedback avoidance and the continuous final route.